NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MASTER OF TECHNOLOGY

COMPARATIVE PERFORMANCE ANALYSIS OF NAIVE STRING MATCHING AND KNUTH–MORRIS–PRATT (KMP) ALGORITHMS ON MULTILINGUAL WIKIPEDIA TITLES

Subject: Advanced Data Structures & Algorithms

Code: 25MCS101

Group No 06

Submitted By

SUVAN SHENOY – NU26CSE21

TANUSHREE PANCHAXARAYYA HIREMATH - NU26CSE22

VASANTH PAI M D - NU26CSE23

VENZIL PAIS-NU26CSE24

Under the guidance of

DR. JYOTHI SHETTY

Professor Department of CSE NMAMIT, NITTE

Date of Submission : 11-11-2026

ABSTRACT

String matching is a fundamental operation in text processing. This study compares the Naive string matching algorithm and the Knuth–Morris–Pratt (KMP) algorithm using multilingual Wikipedia title datasets in English, Kannada and Tulu. The algorithms are evaluated using execution time, character comparisons and correctness against Python's reference substring search. The corpus benchmark used a fixed set of 16 patterns per language and tested these patterns against multiple title-sample sizes. Across the recorded workloads, 2,801,152 algorithm validations passed, corresponding to 100% agreement with the reference results. In the final recorded run, Naive search had lower measured batch execution time than KMP in all ten language/sample-size workloads. This is an observation about the tested workloads, not a claim that Naive has better asymptotic complexity. KMP retains a linear worst-case bound, while Naive has a quadratic worst-case bound in text and pattern lengths. Category-level results show that Naive's search time and character comparisons per pair fall as patterns become longer, while KMP's remain nearly constant. The report discusses the effect of preprocessing, input characteristics and experimental limitations.

Keywords— String matching, Naive algorithm, Knuth–Morris–Pratt, KMP, Longest Prefix Suffix, multilingual text, Unicode NFC, Wikipedia titles, character comparisons, execution time, benchmarking.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Abstract, Table of Contents, List of Figures, List of Tables, Abbreviations and Notation	
1	Introduction	1-2
2	Theoretical Background and Related Technical Concepts	3-4
3	Dataset, Tools and Experimental Environment	5-6
4	Algorithm Design and Implementation	7-8
5	Experimental Methodology	9-10
6	Experimental Results and Comparative Analysis	11-16
7	Discussion and Answers to Research Questions	17-18
8	Limitations and Threats to Validity	19
9	Conclusion and Future Scope	20
	References	21

LIST OF FIGURES

Figure no	Description	Page No
1	English: median search time per pair by pattern-length category.	13
2	Kannada: median search time per pair by pattern-length category.	14
3	Tulu: median search time per pair by pattern-length category.	14
4	English: average character comparisons per pair by pattern-length category.	15
5	Kannada: average character comparisons per pair by pattern-length category.	15
6	Tulu: average character comparisons per pair by pattern-length category.	16

LIST OF TABLES

Table No	Description	Page No
1	Theoretical time and auxiliary-space complexity.	4
2	Source corpus counts and benchmark sample sizes.	5
3	Sampling and pattern-selection seeds.	6
4	Software tools and their roles.	6
5	Pattern categories used for each language.	9
6	Corpus benchmark configuration and measurement counts.	10
7	Corpus benchmark validation summary.	11
8	Median batch execution time for Naive and KMP search.	11
9	Average character comparisons per title-pattern pair.	12

ABBREVIATIONS AND NOTATION

Term	Meaning
ADSA	Advanced Data Structures and Algorithms
CSV	Comma-Separated Values
KMP	Knuth–Morris–Pratt
LPS	Longest Prefix Suffix (table)
NFC	Unicode Normalization Form C
PNG	Portable Network Graphics
s / µs	Second / microsecond
n	Length of the text (title) being searched
m	Length of the pattern
O(·)	Asymptotic upper bound on running time

CHAPTER 1
INTRODUCTION

1.1 Background

String matching determines whether a pattern occurs within a text and, in an all-occurrences setting, identifies every matching position. It is used in search systems, document processing, text editors and information retrieval. The efficiency of a matching algorithm can depend on the text length, pattern length, character distribution, location of matches and number of repeated partial matches.

Multilingual text adds a representation issue. Kannada and Tulu titles contain Unicode characters, and the text representation must be fixed before patterns are compared with titles. This study therefore uses a single, common text field for all matching experiments, as described in Chapters 2 and 3.

1.2 Problem Statement

This assignment investigates two classical algorithms: Naive string matching and KMP [1], [2]. The Naive method tries successive alignments of the pattern against the text. KMP preprocesses the pattern into a Longest Prefix Suffix (LPS) table and uses that information to avoid rechecking text characters unnecessarily. The comparison must be controlled: both algorithms must search the same titles with the same patterns, and every result must be verified against an independent reference.

1.3 Aim

To implement, validate and experimentally compare the Naive and KMP string matching algorithms on English, Kannada and Tulu Wikipedia titles, considering batch execution time, character comparisons and correctness against Python's reference substring search.

1.4 Objectives

- Prepare reproducible samples of Wikipedia title data in English, Kannada and Tulu at several sample sizes.
- Use the title_nfc field as the common text representation for matching.
- Implement the Naive and KMP algorithms so that each reports matching positions and a character-comparison count.

- Validate every result against Python's reference substring search, including overlapping occurrences.
- Measure batch execution time using a warm-up and ten timing repetitions, and measure KMP LPS preprocessing time separately from search time.
- Relate the observed measurements to the theoretical complexity of the two algorithms.

1.5 Research Questions

1. RQ1: How do the Naive and KMP algorithms compare in measured performance?
2. RQ2: How does pattern length affect execution?
3. RQ3: How many character comparisons does each algorithm perform?
4. RQ4: How do the observations relate to theoretical complexity?

1.6 Scope and Boundaries

The experiment searches for patterns within individual titles. It does not concatenate titles into one text, and it does not treat the task as an exact-title lookup. The experiments use selected samples of the English and Kannada datasets rather than every title; the Tulu dataset is used in full at its largest sample size. The study does not evaluate approximate matching, ranking or language-specific collation.

1.7 Report Organization

Chapter 2 explains the two algorithms, Unicode NFC and their complexity. Chapter 3 describes the datasets, sampling and software environment. Chapter 4 explains the implementations and the correctness checks. Chapter 5 documents the pattern workload, timing and validation procedure. Chapter 6 presents the measurements. Chapters 7 and 8 interpret the results and discuss limitations. Chapter 9 concludes the study and identifies future work.

CHAPTER 2
THEORETICAL BACKGROUND AND RELATED TECHNICAL CONCEPTS

2.1 Exact Substring Matching

Both algorithms studied here are suitable for exact substring matching. Given a text of length n and a pattern of length m, the task is to decide whether the pattern occurs in the text and, in the all-occurrences setting, to report every starting position, including overlapping occurrences. In this project the text is a single Wikipedia title and the pattern is a short string selected from the corpus or constructed to be absent from it.

2.2 Unicode NFC Representation

The title_nfc field is used as the common text representation. Unicode NFC normalization helps ensure that canonically equivalent text is represented consistently before matching. The experiment searches within individual titles using this field; it does not concatenate titles into one text or treat the task as an exact-title lookup.

2.3 Naive String Matching

The Naive string matching algorithm is a direct baseline: for each possible starting position in the text, it compares the pattern characters from left to right. It is straightforward to implement and has low preprocessing overhead. Its worst-case running time is O(nm), where n is the text length and m is the pattern length, because it may compare much of the pattern at many text positions.

2.4 Knuth–Morris–Pratt (KMP) Algorithm

Knuth–Morris–Pratt improves the worst-case search bound by preprocessing the pattern [1], [2]. The LPS value at each pattern position records the length of the longest proper prefix that is also a suffix for the pattern prefix considered. On a mismatch, KMP uses the LPS table to shift the pattern without moving the text index backwards. Its preprocessing cost is O(m), and its search cost is O(n), giving O(n + m) overall time.

2.5 Complexity Comparison

TABLE I. THEORETICAL TIME AND AUXILIARY-SPACE COMPLEXITY

Algorithm	Preprocessing time	Search time (worst case)	Overall worst-case time	Additional structure
Naive	None	O(nm)	O(nm)	None
KMP	O(m)	O(n)	O(n + m)	LPS table of m entries

The table is a theoretical reference. The experiment does not assume that measured results must follow the asymptotic bounds on every finite workload.

2.6 Theory versus Practical Performance

Both algorithms are exact, but their practical performance can differ from their asymptotic bounds because real inputs are finite and may contain short patterns, early matches, frequent mismatches or limited opportunities for KMP to reuse prefix information. KMP's advantage is designed to appear in difficult mismatch patterns that cause the Naive method to repeat work; a finite workload may not trigger those cases. Python implementation overhead and the cost of constructing and using the LPS table can also influence measured time.

CHAPTER 3
DATASET, TOOLS AND EXPERIMENTAL ENVIRONMENT

3.1 Dataset

The experiments use Wikipedia title datasets in three languages, supplied as CSV files for the ADSA group assignment [3]. The source datasets contain Wikipedia title records. Table II lists the original title counts and the sample sizes used.

TABLE II. SOURCE CORPUS COUNTS AND BENCHMARK SAMPLE SIZES

Language	Original title count	Dataset file	Sample sizes used
English	19,264,252	english_titles.csv	1,000; 5,000; 10,000; 25,000
Kannada	47,392	kannada_titles.csv	1,000; 5,000; 10,000; 25,000
Tulu	4,536	tulu_titles.csv	1,000; 4,536

The project uses the title_nfc column for matching. The reported sample sizes are 1,000, 5,000, 10,000 and 25,000 for English and Kannada, where available; Tulu uses 1,000 and 4,536 titles. The samples and their configuration are recorded in the project's sampling outputs and manifest. Because the English and Kannada samples are subsets of much larger sources, the results should not be read as measurements over the complete corpora.

3.2 Sampling Strategy

The dataset sampling implementation uses Reservoir Sampling with a base seed of 42. The language-specific reservoir seeds are 1042 for Tulu, 2042 for Kannada and 3042 for English. Smaller samples are selected from the maximum-size reservoir using separate subsample seeds recorded in the sampling manifest. Pattern selection uses its own seed, 2406.

TABLE III. SAMPLING AND PATTERN-SELECTION SEEDS

Parameter	Value
Base sampling seed	42
Tulu reservoir seed	1042
Kannada reservoir seed	2042
English reservoir seed	3042

Parameter	Value
Pattern-selection seed	2406
Subsample seeds	Recorded in the sampling manifest

3.3 Software and Environment

The recorded environment is Windows with Python 3.13.14, executed through Visual Studio Code. The project uses pandas for CSV handling, NumPy for numerical operations where required, and Matplotlib for graph generation [4]–[7]. Package versions can affect performance, so the exact installed versions should be taken from the environment.txt file submitted with the project.

TABLE IV. SOFTWARE TOOLS AND THEIR ROLES

Tool / library	Role in the project	Where it is used
Python 3.13.14	Primary implementation language	Algorithms, sampling and benchmarking
pandas	CSV handling and tabular summaries	Dataset loading, result tables
NumPy	Numerical operations where required	Analysis
Matplotlib	Graph generation	Performance graphs
Visual Studio Code	Development and execution environment	Project development
Windows	Operating system of the recorded run	Benchmark execution

CHAPTER 4
ALGORITHM DESIGN AND IMPLEMENTATION

4.1 Common Search Contract

Both methods accept a pattern and a single title and report the matching position(s) together with a count of character comparisons. Each pattern is searched separately within every title, which creates title-pattern pairs and avoids cross-title matches. The same contract allows the results to be compared directly with a reference substring search.

4.2 Naive Implementation

The Naive implementation tries each possible alignment of the pattern against the title. At each alignment it compares characters from left to right and stops the alignment at the first mismatch; a complete match is recorded as an occurrence. The implementation reports the matching position(s) and the character-comparison count. It needs no preprocessing and keeps no additional structure.

Pseudocode (outline): FOR each start position s from 0 to n − m; compare pattern[j] with text[s + j] for j = 0, 1, …; IF all m characters match, record s; END FOR.

4.3 KMP Implementation

The KMP implementation uses an LPS construction routine to preprocess the pattern, then scans the title once, using the LPS table to shift the pattern after a mismatch without moving the text index backwards. It reports matching positions and character comparisons. KMP preprocessing time is measured separately from search time.

Pseudocode (outline): BUILD lps[0..m − 1]; SET i = 0 and j = 0; WHILE i < n; IF text[i] equals pattern[j], advance both indices and record an occurrence when j = m; ELSE IF j > 0, set j = lps[j − 1]; ELSE advance i; END WHILE.

4.4 Correctness Checking

The benchmark checks the results of both algorithms against Python's reference substring search [4], including overlapping occurrences. Every title-pattern pair is checked for each algorithm, so the number of validations is twice the number of pairs. A disagreement with the reference would indicate an implementation error; none was observed in the recorded run (Chapter 6).

4.5 Why the Implementations Are Comparable

Both algorithms search the same sampled titles with the same fixed pattern set for each language, and both are checked against the same reference. A batch consists of the selected patterns searched against all titles in a particular sample, and the same warm-up and ten-repetition timing policy is applied. These controls reduce avoidable differences unrelated to the algorithms themselves.

CHAPTER 5
EXPERIMENTAL METHODOLOGY

5.1 Overall Experimental Design

The project follows a controlled, repeatable procedure. Title data are sampled with fixed seeds, patterns are selected with a fixed seed, both algorithms are run on every title-pattern pair, and results are validated against a reference search. Timing and comparison counts are then summarized and plotted.

5.2 Pattern Workload

For the corpus benchmark, 16 patterns are selected per language: two patterns from each combination of matching or non-matching status and four pattern-length categories (short, medium, long and very long). The same selected pattern set is applied across the sample sizes for that language. Each pattern is searched separately within every title, so the number of title-pattern pairs for a sample equals 16 times the number of titles.

TABLE V. PATTERN CATEGORIES USED FOR EACH LANGUAGE

Match status	Length category	Patterns per category	Subtotal
Matching	Short, medium, long, very long	2	8
Non-matching	Short, medium, long, very long	2	8
Total per language			**16**

The exact length boundaries of the four categories are recorded with the benchmark outputs (pattern_length_analysis.csv) and are not reproduced in this report.

5.3 Benchmark Configurations

The final recorded corpus benchmark covers ten workload configurations: four English sample sizes, four Kannada sample sizes and two Tulu sample sizes. In total there are 1,400,576 title-pattern pairs, and since both algorithms are checked for each pair, there are 2,801,152 validations.

TABLE VI. CORPUS BENCHMARK CONFIGURATION AND MEASUREMENT COUNTS

Item	Value
Languages	English, Kannada, Tulu
Workload configurations	10 (English 4, Kannada 4, Tulu 2)
Patterns per language	16
Title-pattern pairs (total)	1,400,576
Algorithms compared	Naive and KMP
Algorithm validations (total)	2,801,152
Warm-up	Performed before timing
Timing repetitions	10 (median reported)

5.4 Metrics and Timing Procedure

The principal metrics are (i) batch execution time, (ii) character comparisons, (iii) KMP LPS preprocessing time, and (iv) correctness against the reference search. The corpus benchmark performs a warm-up and uses ten timing repetitions; the reported batch times are the recorded median measurements. A batch consists of the selected patterns searched against all titles in a particular sample. KMP preprocessing time is measured separately from search time.

5.5 Validation Procedure

Every title-pattern pair is validated for both algorithms against Python's reference substring search, including overlapping occurrences. The campaign is considered valid only if all validations agree with the reference.

5.6 Reproducibility Controls

Reproducibility controls include fixed base and language-specific seeds, a recorded pattern-selection seed, a sampling manifest, the saved result tables and graphs, and an environment.txt file recording installed package versions. Appendix A lists the evidence files produced by the experiment.

CHAPTER 6
EXPERIMENTAL RESULTS AND COMPARATIVE ANALYSIS

6.1 Benchmark Completion and Validation

The final recorded corpus benchmark completed ten workload configurations: four English sample sizes, four Kannada sample sizes and two Tulu sample sizes. In total, there were 1,400,576 title-pattern pairs. Both algorithms were checked for each pair, giving 2,801,152 validations. All validations passed.

TABLE VII. CORPUS BENCHMARK VALIDATION SUMMARY

Validation item	Expected	Observed	Result
Workload configurations completed	10	10	PASS
Title-pattern pairs	1,400,576	1,400,576	PASS
Algorithm validations (Naive + KMP)	2,801,152	2,801,152 passed	PASS
Agreement with reference search	100%	100%	PASS

6.2 Batch Execution Time

Table VIII reports the median batch execution time for each workload. The last column, the ratio of KMP time to Naive time, is computed from the tabulated medians and is therefore subject to their four-decimal rounding.

TABLE VIII. MEDIAN BATCH EXECUTION TIME FOR NAIVE AND KMP SEARCH

Language	Titles	Pairs	Naive time (s)	KMP time (s)	KMP / Naive
English	1,000	16,000	0.0232	0.0275	1.19
English	5,000	80,000	0.1076	0.1354	1.26
English	10,000	160,000	0.2241	0.2726	1.22
English	25,000	400,000	0.5452	0.6680	1.23
Kannada	1,000	16,000	0.0279	0.0395	1.42
Kannada	5,000	80,000	0.1448	0.2038	1.41
Kannada	10,000	160,000	0.2986	0.3956	1.32
Kannada	25,000	400,000	0.7177	1.0143	1.41

Language	Titles	Pairs	Naive time (s)	KMP time (s)	KMP / Naive
Tulu	1,000	16,000	0.0234	0.0366	1.56
Tulu	4,536	72,576	0.1032	0.1604	1.55

In all ten configurations, the measured Naive batch time was lower than the measured KMP batch time. Batch time increased with sample size for both algorithms, as more title-pattern pairs were processed.

Kannada workloads took longer than English or Tulu workloads at comparable sample sizes in this run; this may reflect differences in title lengths and character content, but the available aggregate table alone cannot isolate the cause.

6.3 Naive versus KMP

The ratio of KMP time to Naive time (Table VIII, last column) ranged from 1.19 to 1.56 across the ten configurations: 1.19 to 1.26 for English, 1.32 to 1.42 for Kannada and about 1.55 to 1.56 for Tulu. Within each language the ratio stayed in a narrow band as the sample size grew, which is consistent with both algorithms processing proportionally more title-pattern pairs at larger sample sizes. These are observations about the tested workloads and do not imply that Naive is faster for arbitrary inputs.

6.4 Character Comparisons

The comparison-count summary reported the average number of comparisons per title-pattern pair for each language (Table IX). These averages describe the measured workload and do not establish that KMP always performs more comparisons for arbitrary inputs.

TABLE IX. AVERAGE CHARACTER COMPARISONS PER TITLE-PATTERN PAIR

Language	Naive	KMP	KMP / Naive
English	15.05	21.65	1.44
Kannada	9.70	15.19	1.57
Tulu	7.12	12.79	1.80

For every language, KMP's average comparison count was higher than Naive's on this workload, in the same direction as the batch-time results. The KMP/Naive column is computed from the reported averages.

6.5 Effect of Pattern Length

The 16 patterns per language are grouped into short, medium, long and very-long categories (Table V). Figs. 2–7 report category-level results for each language. The values quoted below are approximate readings from the graphs; the exact figures are in pattern_length_analysis.csv.

6.5.1 Search execution time

Figs. 2–4 show the median search time per title-pattern pair, in microseconds, for each pattern-length category.

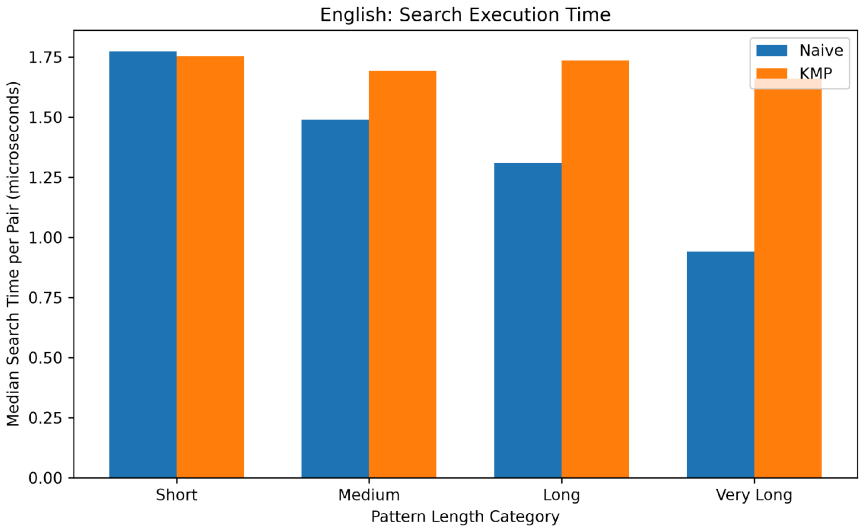


Fig. 1. English: median search time per pair by pattern-length category.

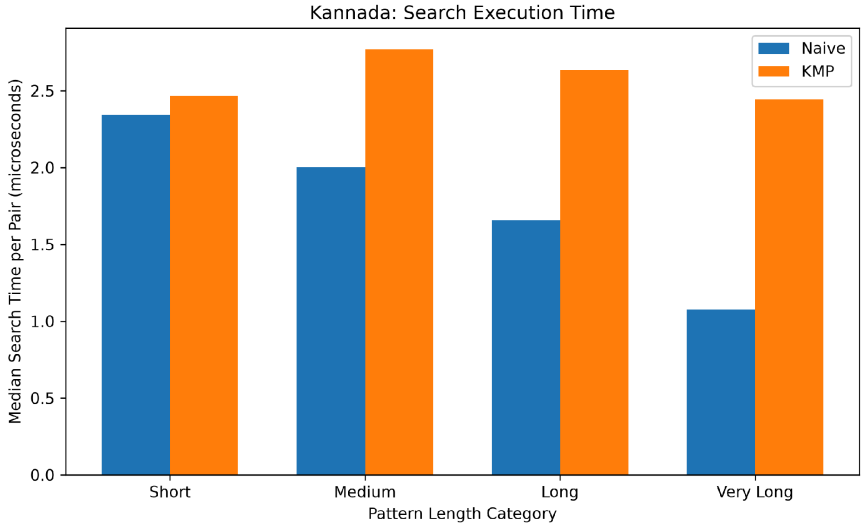


Fig. 2. Kannada: median search time per pair by pattern-length category.

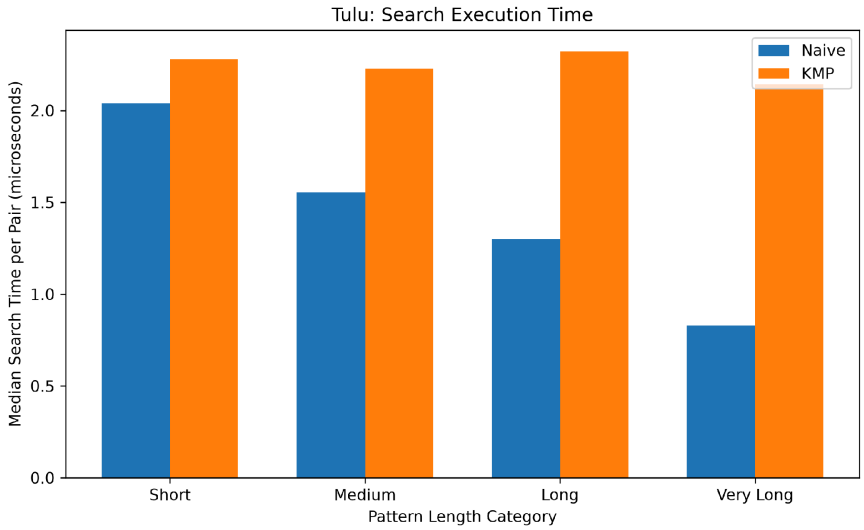


Fig. 3. Tulu: median search time per pair by pattern-length category.

In all three languages the Naive search time falls as the pattern becomes longer. From the short to the very-long category it drops from roughly 2.0 to 0.8 µs in Tulu, from 2.3 to 1.1 µs in Kannada and from 1.8 to 0.9 µs in English. The KMP search time stays nearly flat across the categories, at roughly 2.1–2.3 µs in Tulu, 2.4–2.8 µs in Kannada and 1.7 µs in English. Consequently the gap between the two algorithms widens with pattern length. For short patterns the two are close: in Kannada KMP is only slightly slower, and in English short patterns are the one case where the two are essentially equal (about 1.77 µs for Naive and 1.75 µs for KMP). In every other category of every language, KMP is slower than Naive.

6.5.2 Character comparisons

Figs. 5–7 show the average number of character comparisons per title-pattern pair for each category.

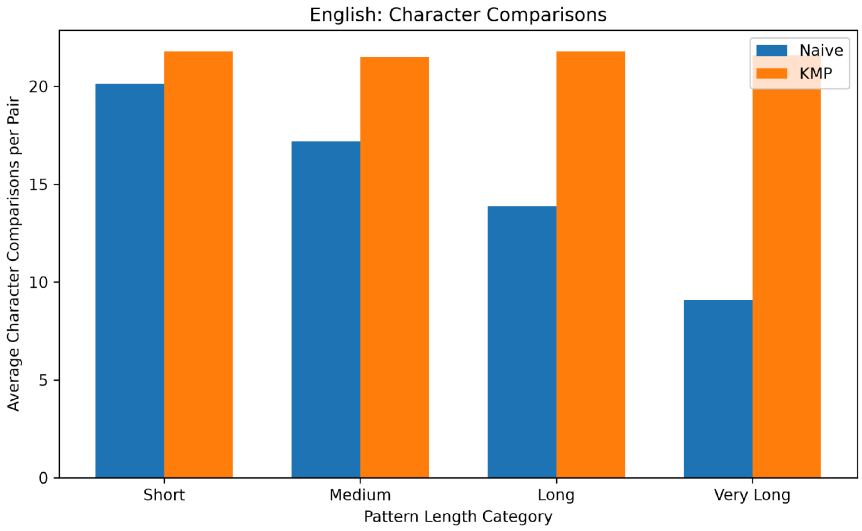


Fig. 4. English: average character comparisons per pair by pattern-length category.

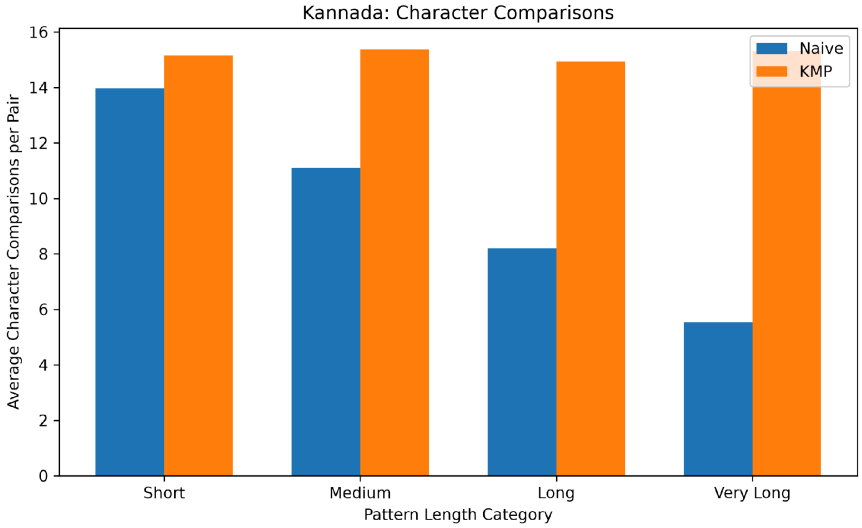


Fig. 5. Kannada: average character comparisons per pair by pattern-length category.

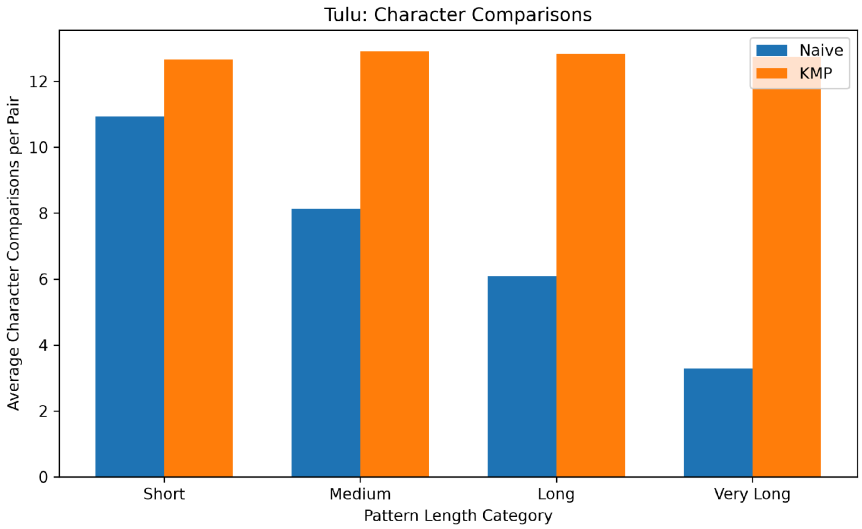


Fig. 6. Tulu: average character comparisons per pair by pattern-length category.

The comparison counts follow the same pattern. Naive's average falls steadily with pattern length: from roughly 20 to 9 comparisons in English, from 14 to 5.5 in Kannada and from 11 to 3 in Tulu. KMP's average is almost constant across categories, at roughly 22 in English, 15 in Kannada and 13 in Tulu. Averaging the four categories gives values consistent with the per-language averages in Table IX (for example, about 15 for Naive in English and about 7 in Tulu).

A plausible contributor is that a longer pattern leaves fewer possible alignments inside a short title, so Naive has fewer positions to try, whereas KMP scans the title once regardless of pattern length. This explanation is a hypothesis consistent with the graphs; the aggregate results do not isolate title length or match position as separate factors.

CHAPTER 7
DISCUSSION AND ANSWERS TO RESEARCH QUESTIONS

7.1 RQ1: Measured Performance of Naive and KMP

The measured results show that Naive search was faster for the tested corpus workloads: its batch time was lower than KMP's in all ten language/sample-size configurations, by a factor of roughly 1.2 to 1.6 (Table VIII). A plausible explanation is that the selected titles and patterns often allow the simple algorithm to finish with relatively few comparisons, while KMP incurs the additional cost of constructing and using the LPS table. For short patterns and short title strings, preprocessing overhead can be significant relative to the amount of search work.

7.2 RQ2: Effect of Pattern Length

Pattern length has a clear effect on Naive but little effect on KMP in these workloads. As patterns lengthen from the short to the very-long category, Naive's search time and comparison count per pair fall substantially in all three languages, while KMP's stay nearly constant (Figs. 2–7). The advantage of Naive therefore grows with pattern length: it is small for short patterns, where the two algorithms are close and, for English, essentially equal, and largest for very long patterns. This describes the sampled titles and the 16 fixed patterns per language; it should not be extrapolated to patterns or texts that provoke repeated partial matches.

7.3 RQ3: Character Comparisons

On average, Naive performed fewer character comparisons per title-pattern pair than KMP in each language: 15.05 versus 21.65 for English, 9.70 versus 15.19 for Kannada and 7.12 versus 12.79 for Tulu. These averages describe the measured workload and do not establish that KMP always performs more comparisons for arbitrary inputs.

7.4 RQ4: Relation to Theoretical Complexity

This practical observation does not contradict KMP's theoretical advantage. The Naive algorithm has worst-case time O(nm), because it may compare much of the pattern at many text positions. KMP takes O(m) time to construct the LPS table and O(n) time to scan the text, so its total worst-case time is O(n + m) [1], [2]. KMP is designed to prevent repeated

work in difficult mismatch patterns. A finite workload of short titles and short patterns may not trigger those worst cases, so the measured ordering cannot be used to rank the algorithms asymptotically.

7.5 Sample Size and Language

The benchmark shows that total batch time grows as the number of titles increases. Since the same 16 patterns are applied to each title, increasing the number of titles increases the number of title-pattern pairs proportionally. Dividing batch time by the number of pairs (Appendix B, Table X) gives a roughly stable cost per pair within each language. Language-level differences should be interpreted carefully: Unicode code points, title lengths and character distributions can affect comparisons and runtime, and the current aggregate results do not isolate these factors independently.

7.6 Practical Considerations

KMP preprocessing and search are separate costs and must be combined appropriately when assessing the total cost for a one-off pattern; if a pattern is reused many times, preprocessing may be amortized. For workloads dominated by short patterns and short strings, as in this study, the simple Naive algorithm was competitive and faster in practice. Where patterns or texts are highly repetitive and could trigger repeated partial matches, KMP's linear worst-case bound is the safer guarantee. The choice should consider the expected pattern and text characteristics as well as measured time.

CHAPTER 8
LIMITATIONS AND THREATS TO VALIDITY

8.1 Sample Coverage

The experiments use selected samples rather than every title in the English and Kannada datasets. The results therefore do not establish performance over the complete source corpora.

8.2 Execution Environment

Measured time depends on the computer, Python runtime, background processes and measurement conditions. The recorded environment is Windows with Python 3.13.14 run through Visual Studio Code. The numbers should be read as measurements from that environment, not as hardware-independent constants.

8.3 Pattern Workload

The benchmark uses a finite set of 16 patterns per language, so its results may not represent all possible patterns or text distributions.

8.4 Granularity of the Results

Aggregate timing values do not explain the contribution of individual pattern lengths, title lengths or match positions. The category-level graphs are summarized from approximate readings and should be checked against the recorded CSV values.

8.5 Preprocessing and Search Cost

KMP preprocessing and search are separate costs and must be combined appropriately when assessing total cost for a one-off pattern. If a pattern is reused many times, preprocessing may be amortized.

8.6 Implementation Overhead and Derived Ratios

Python implementation overhead can influence measured performance and should not be confused with the algorithms' asymptotic complexity. The KMP/Naive ratios in this report are computed from rounded tabulated values and are descriptive only; the study does not report a statistical test of the differences.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project compared Naive and KMP string matching on English, Kannada and Tulu Wikipedia title samples. Both implementations agreed with the reference substring search for all 2,801,152 algorithm validations in the corpus benchmark. Naive had lower measured batch execution time in all ten recorded configurations, while KMP retained the stronger theoretical worst-case bound of O(n + m), compared with O(nm) for Naive. The results demonstrate that practical performance depends on workload characteristics and preprocessing overhead as well as asymptotic complexity.

These findings are limited to the sampled titles, the fixed 16-pattern sets, the Python implementations and the recorded execution environment. Pattern length strongly affected Naive, whose time and comparisons per pair fell as patterns lengthened, but had little effect on KMP.

9.2 Future Scope

- Test whether the pattern-length trends persist for larger pattern sets and for patterns that provoke repeated partial matches.
- Extend the experiment to larger samples of the English and Kannada datasets.
- Use larger and more varied pattern sets, including repetitive patterns that can trigger the worst-case behaviour of the Naive algorithm.
- Repeat the benchmark in other environments or implementation languages to separate Python overhead from algorithmic effects.
- Report KMP LPS preprocessing time alongside search time for single-use and repeated-use patterns.

REFERENCES

[1]D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast pattern matching in strings,” *SIAM J. Comput.*, vol. 6, no. 2, pp. 323–350, 1977.

[2]J. H. Morris and V. R. Pratt, “A linear pattern-matching algorithm,” Univ. California, Berkeley, CA, USA, Tech. Rep. 40, 1970.

[3]Multilingual Wikipedia title datasets supplied for the ADSA group assignment (English, Kannada and Tulu).

[4]Python Software Foundation, “Python 3 documentation.” [Online]. Available: https://docs.python.org/3/

[5]pandas Development Team, “pandas documentation.” [Online]. Available: https://pandas.pydata.org/docs/

[6]NumPy Developers, “NumPy documentation.” [Online]. Available: https://numpy.org/doc/

[7]Matplotlib Development Team, “Matplotlib documentation.” [Online]. Available: https://matplotlib.org/stable/